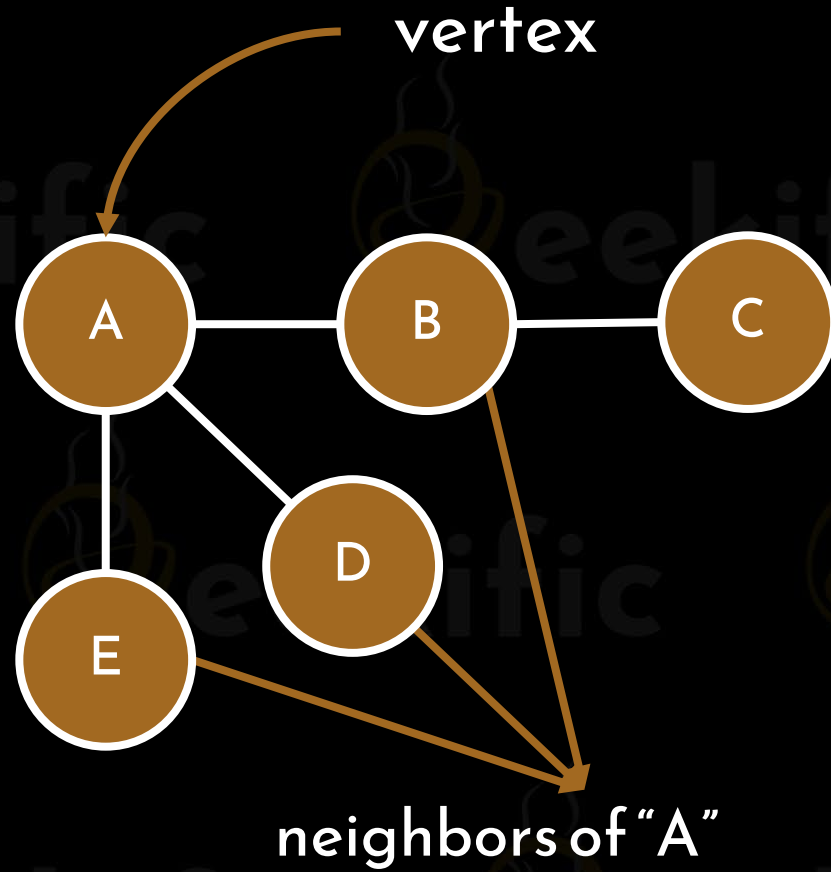


# Graphs

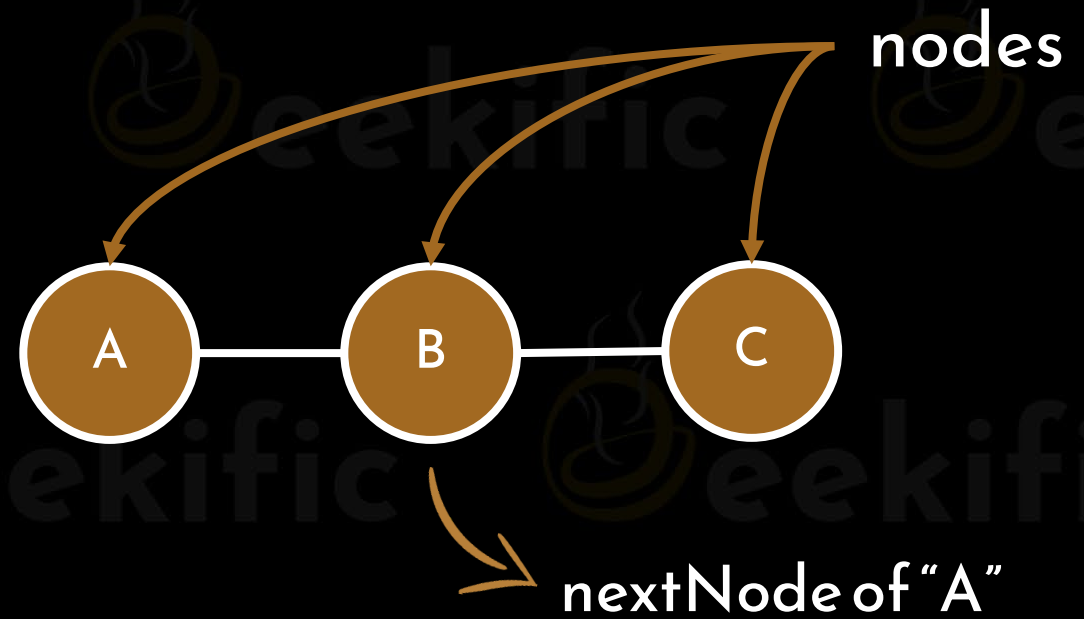
types, representations, trees, spanning trees  
& strongly connected components

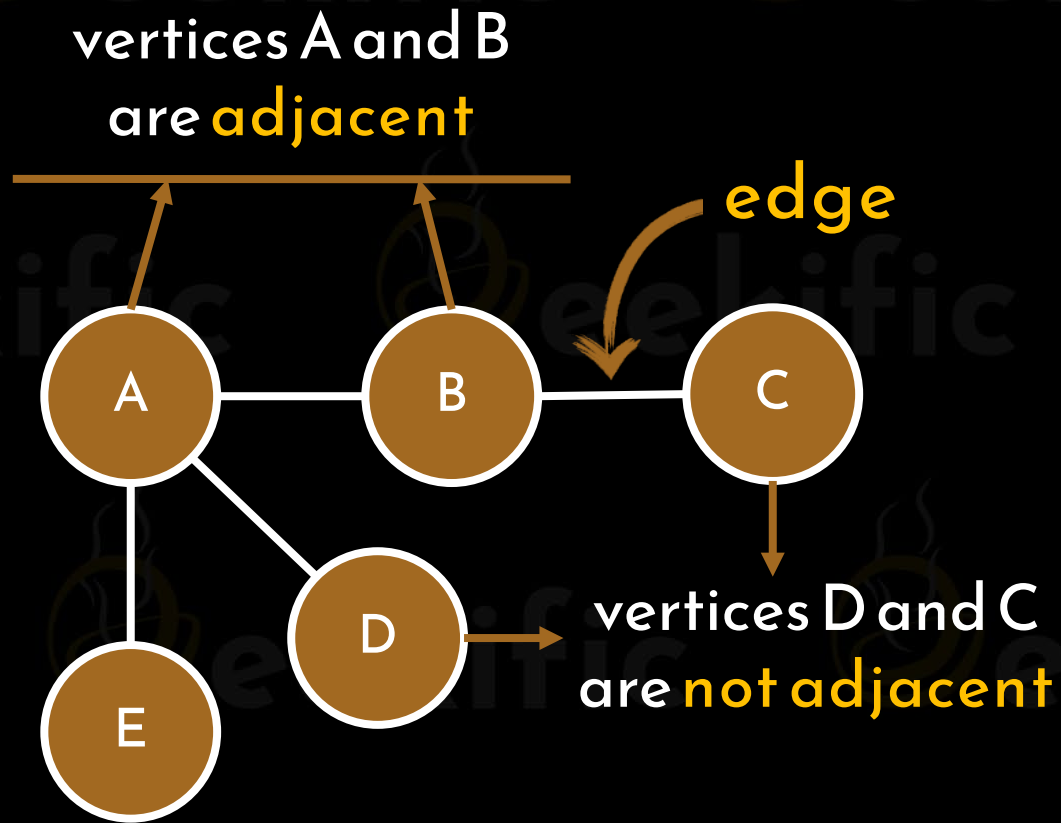


# Graph



# LinkedList





**edge** connecting A and B =  $(A, B)$

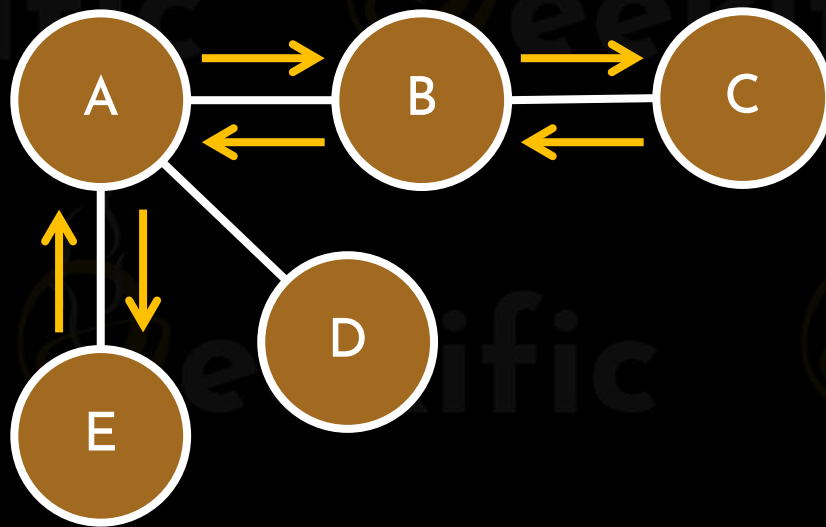
**edge** connecting B and A =  $(B, A)$

**edge** connecting C and B =  $(C, B)$



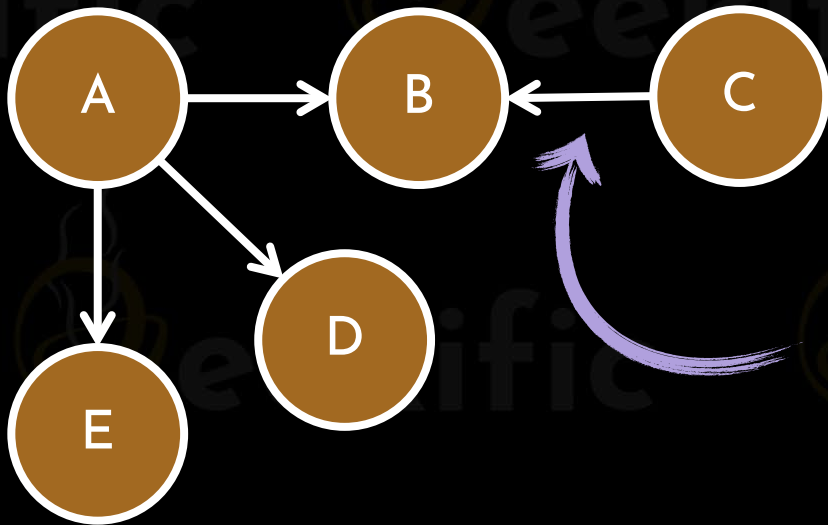
undirected  
graph

if  $(A, B)$  exists then  
 $(B, A)$  also exists



directed graph

if  $(A, B)$  exists there is  
no guarantee that  
 $(B, A)$  also exists



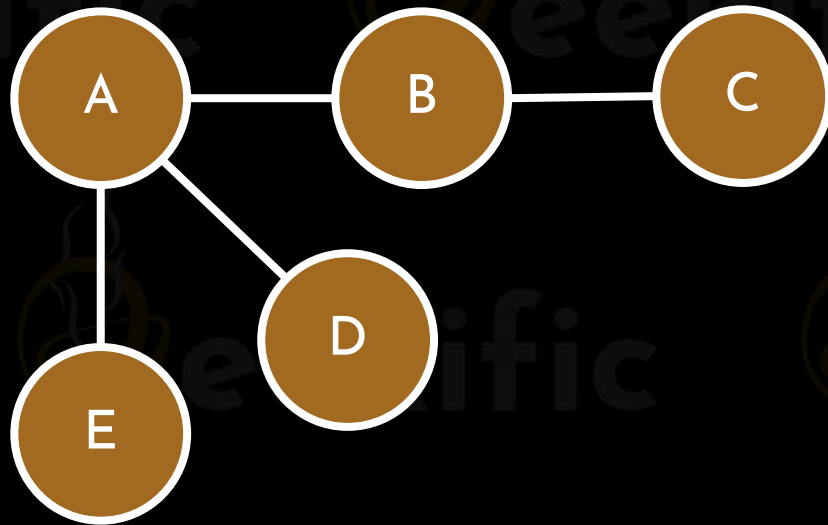
the arrows determine the  
direction in which an edge  
can be traversed



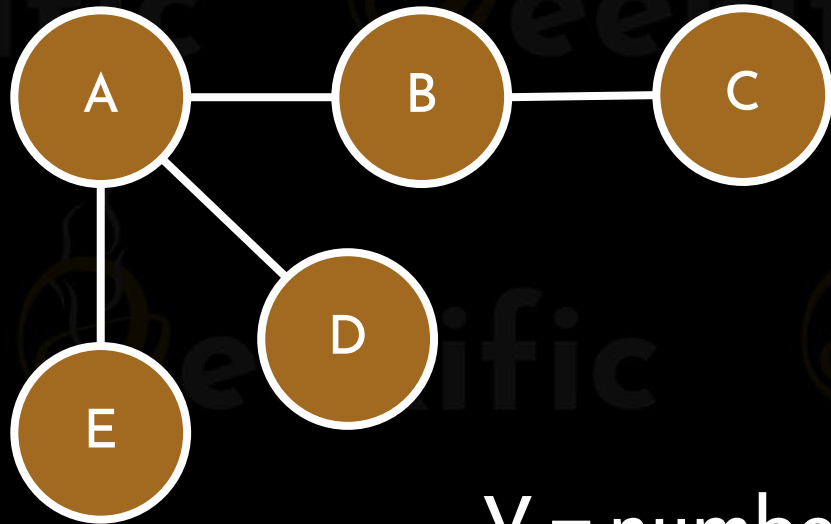
Different ways to  
**represent** a graph:

adjacency **matrix**

adjacency **list**



# adjacency matrix



$V$  = number  
of vertices

edge\*

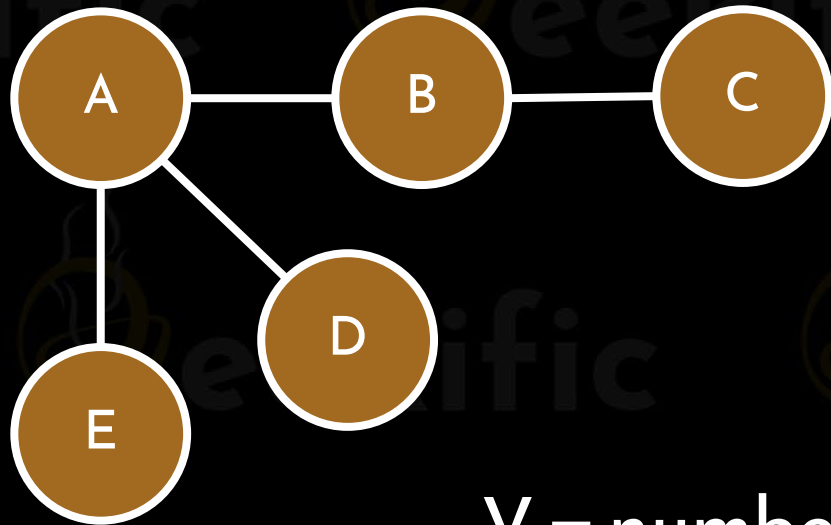
each row and each column  
represents a vertex

|   | A     | B     | C     | D     | E     |
|---|-------|-------|-------|-------|-------|
| A | false | true  | false | true  | true  |
| B | true  | false | true  | false | false |
| C | false | true  | false | false | false |
| D | true  | false | false | false | false |
| E | true  | false | false | false | false |

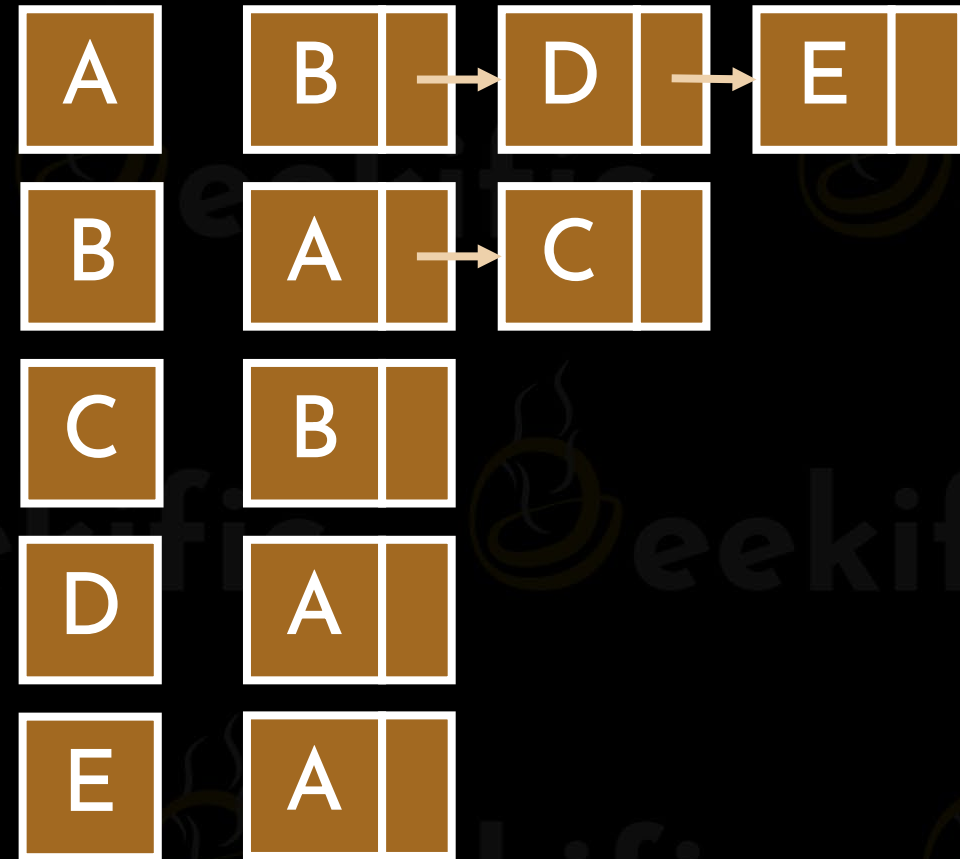


adjacency **list**

a map where the key is our vertex  
and the value a list of its neighbors



$V$  = number  
of vertices



# Adjacency List

if vertices store **integers**, use an **array of lists**  
where array indices represent the vertices

**Maps** or **Hash Tables** are the more **generic form**



# Matrix vs List

the **adjacency matrix** is always **symmetric** for undirected graphs

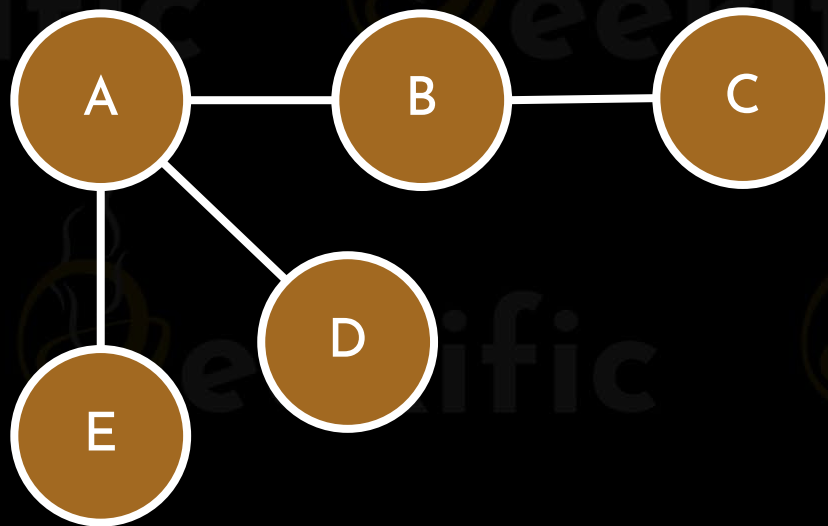
it allows **fast edge lookup** but requires  **$V^2$  space**

the **adjacency list** only stores existing edges making it **more space efficient**



directed graph

undirected graph



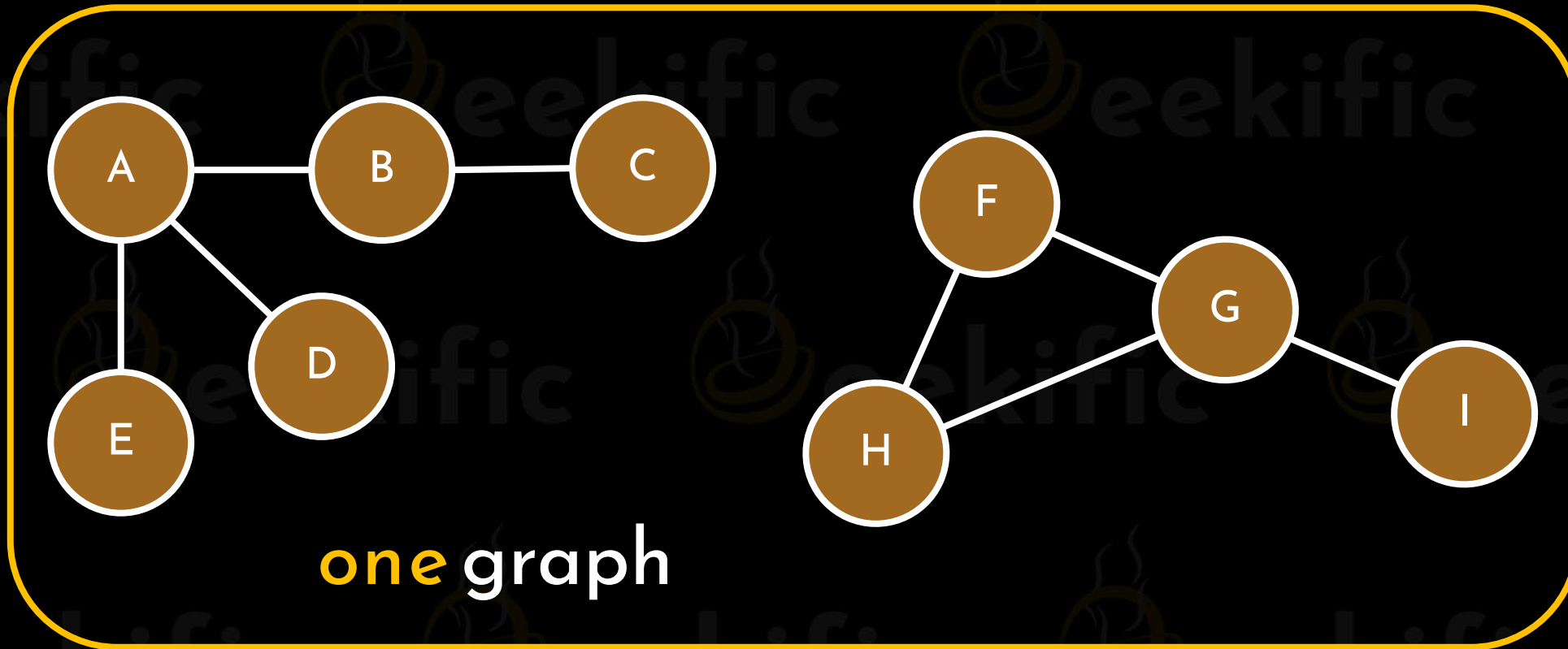
adjacency matrix

adjacency list



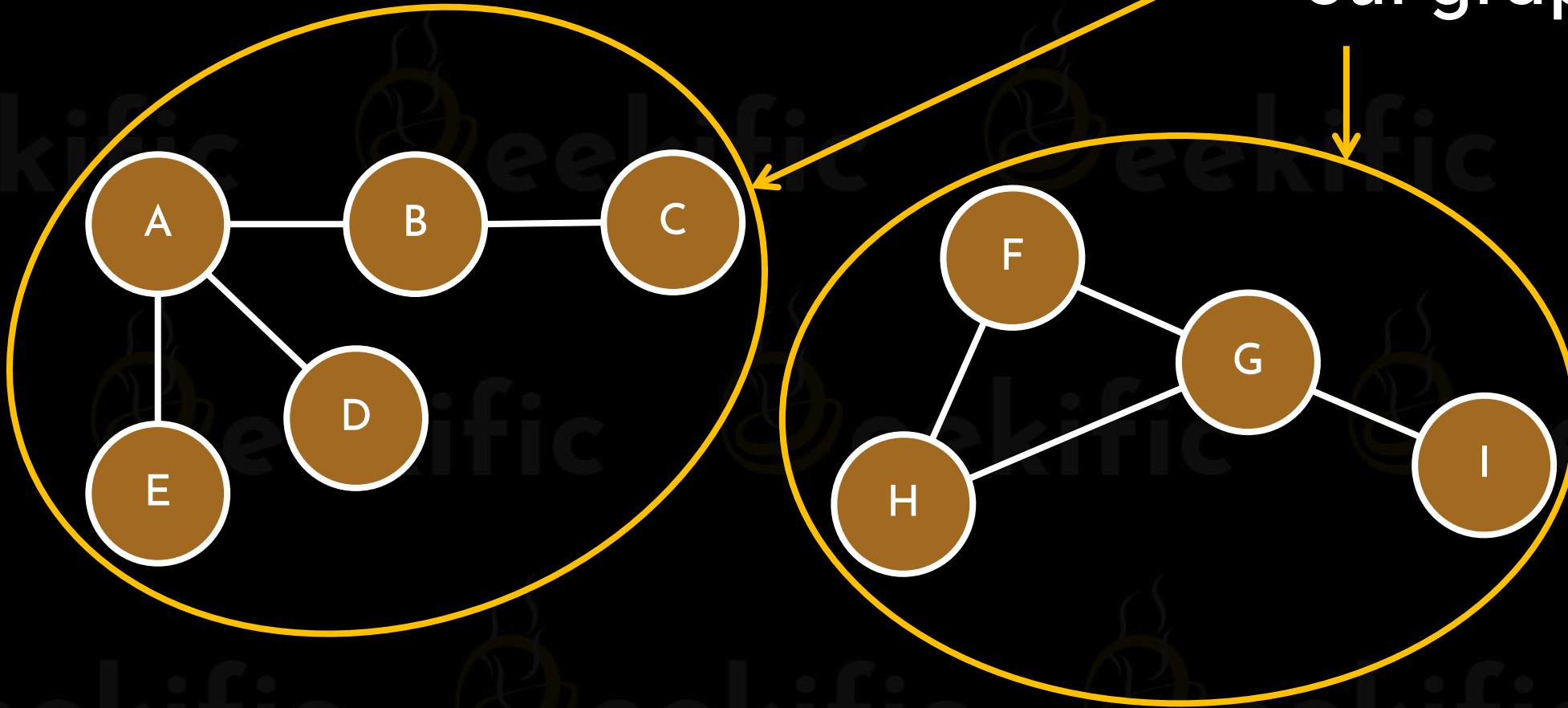
H cannot be reached from A  
nor B, nor C...

**disconnected**  
undirected graph



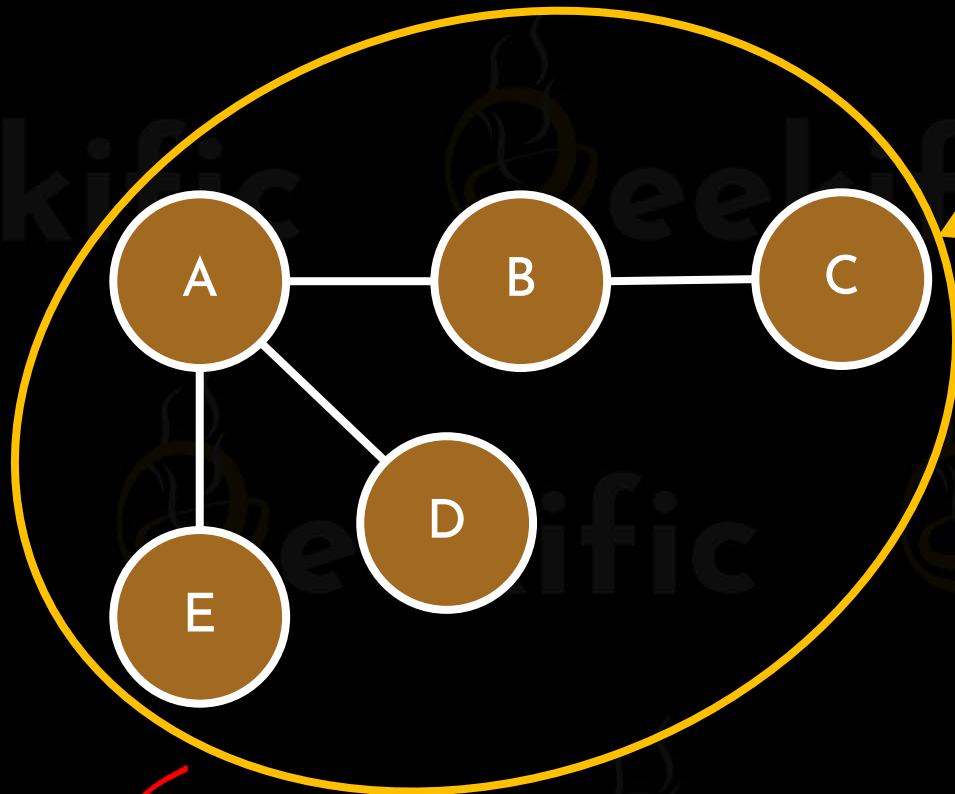
...nor any node present in the first component of the graph

**components** of  
our graph

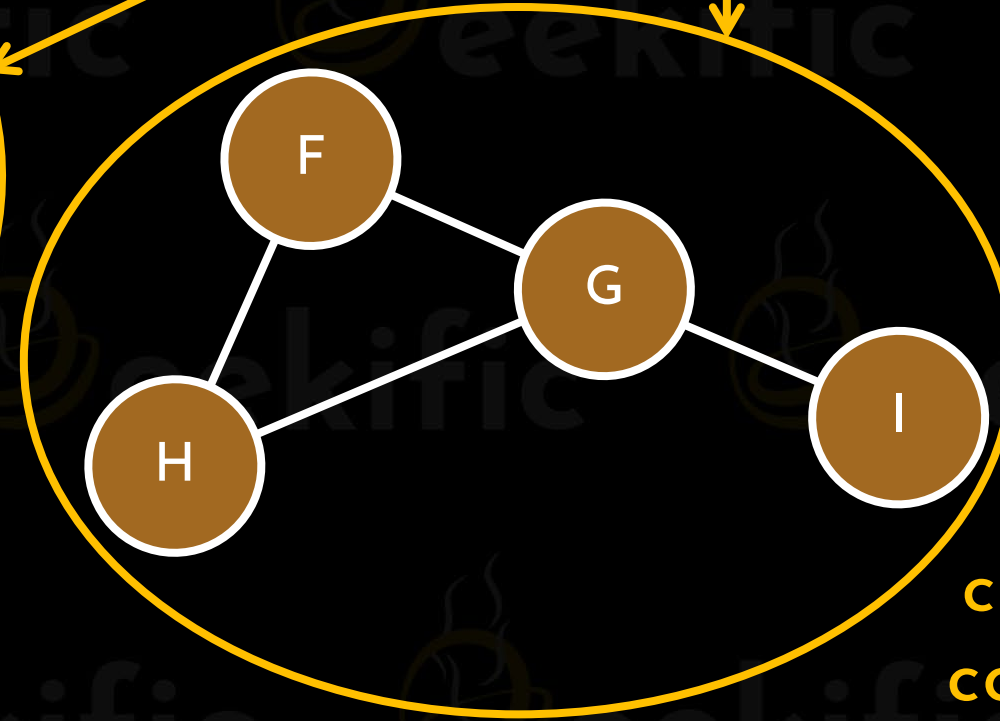


every vertex can be **reached**  
from any other vertex from  
**within** these components

**components** of  
our graph



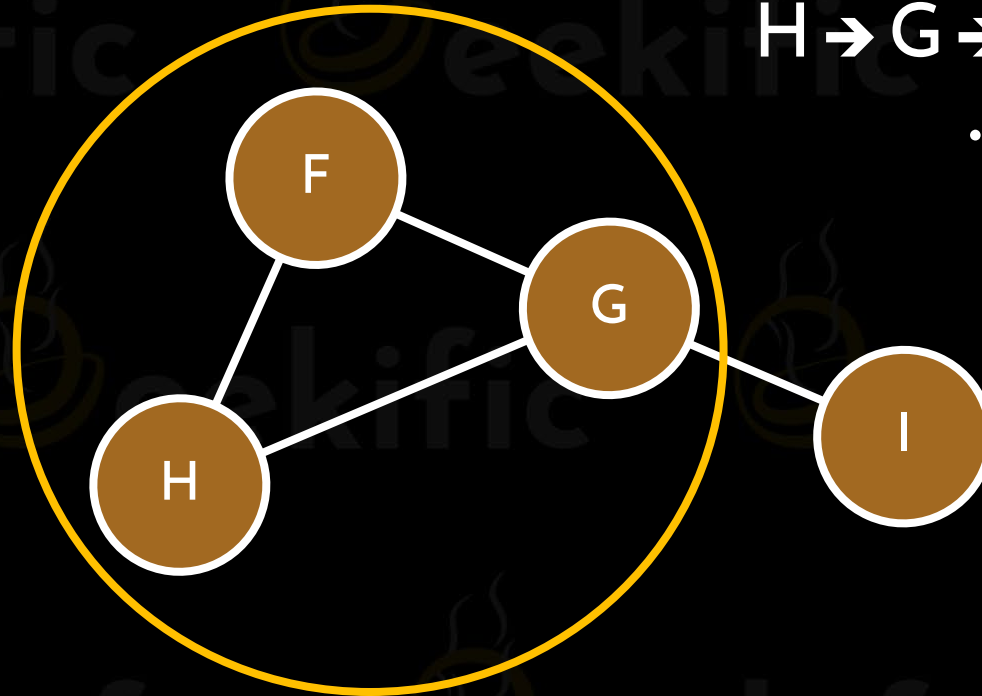
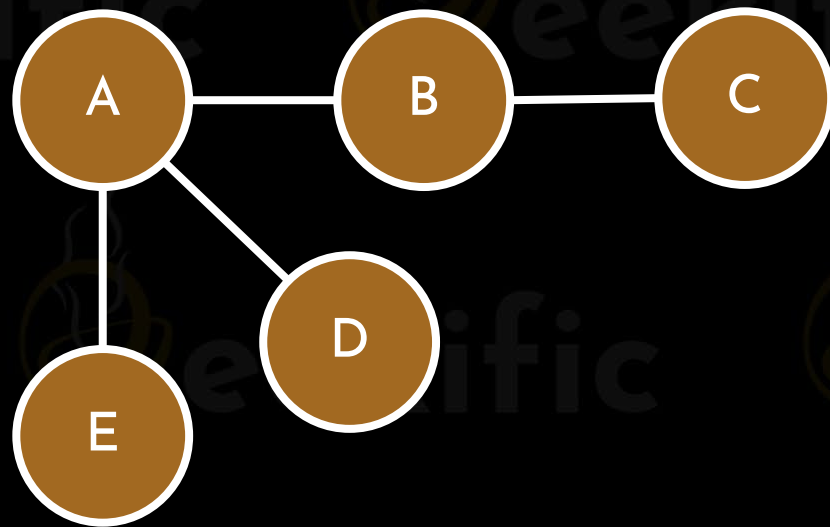
**a connected  
component**



**another  
connected  
component**



a **cycle** is detected if a path can be established where only the **first and the last vertex** are **repeated**



$H \rightarrow F \rightarrow G \rightarrow H$

$F \rightarrow G \rightarrow H \rightarrow F$

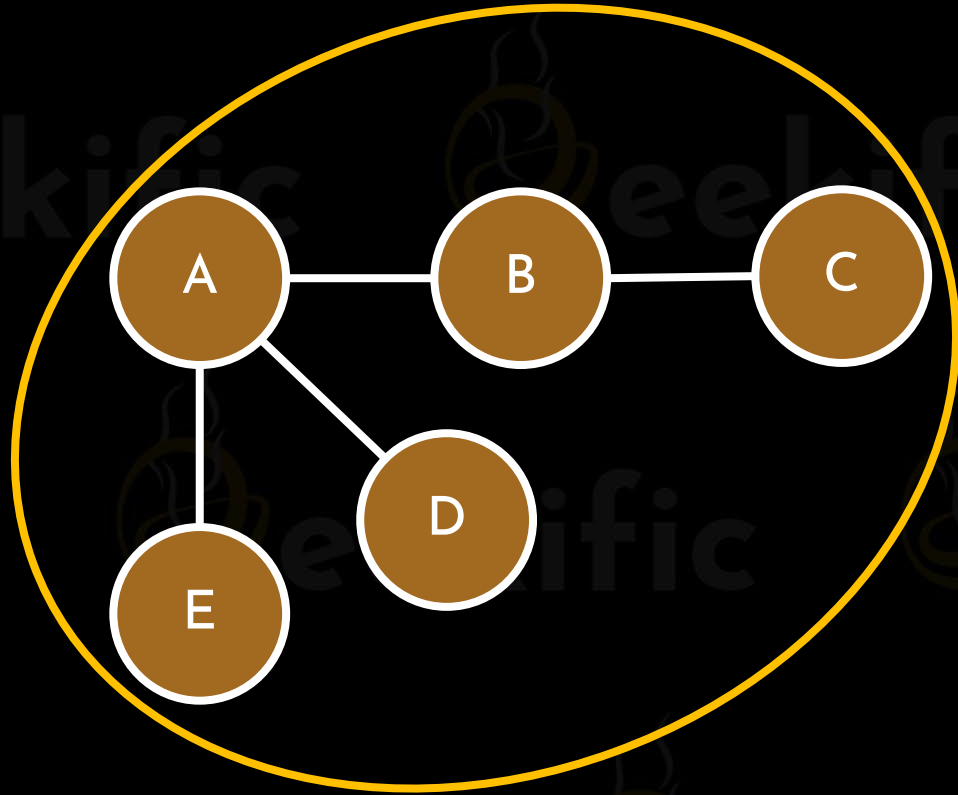
$G \rightarrow F \rightarrow H \rightarrow G$

$H \rightarrow G \rightarrow F \rightarrow H$

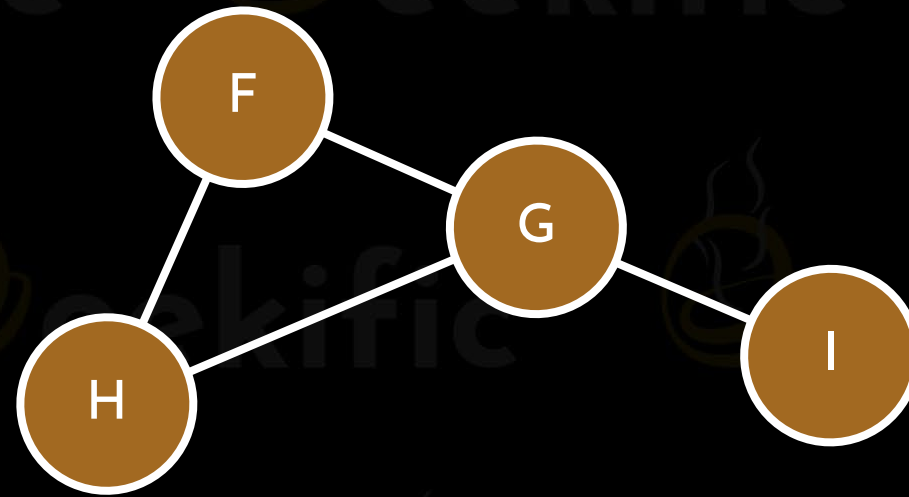
...



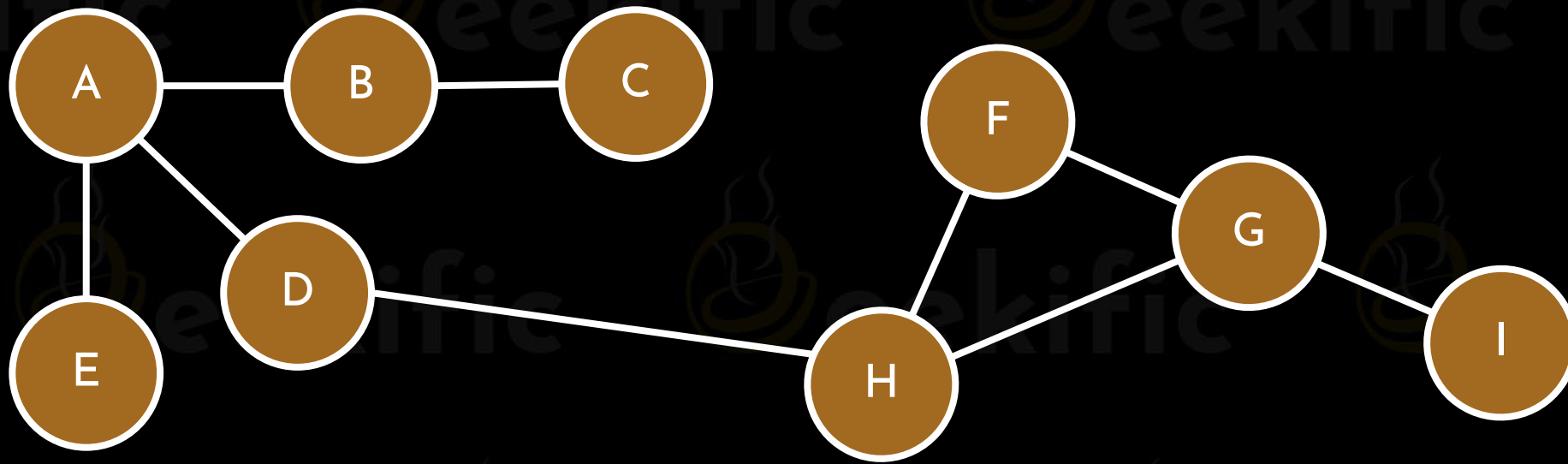
a graph with **no cycles** is  
called an **acyclic** graph



if an acyclic graph is also  
**undirected** and **connected** it  
becomes a **tree**



adding **any edge** between two components  
makes the graph **connected**



# Undirected to Directed

adding **any edge** between two disconnected components makes the graph **connected**

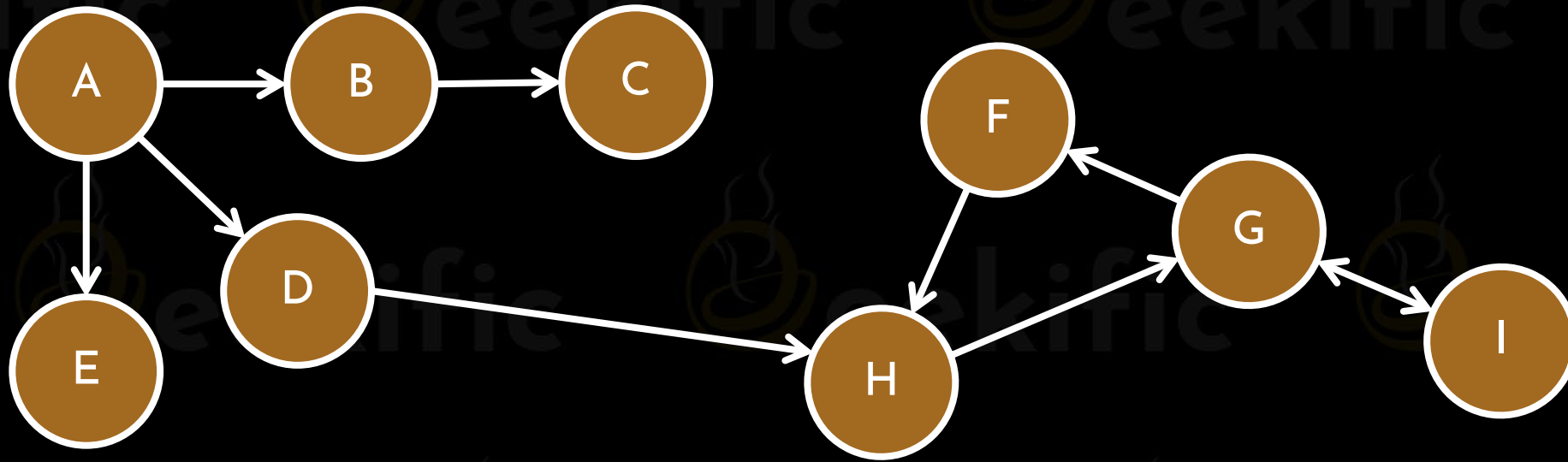
but when we convert to a **directed graph** **connected** splits into two definitions:

**weakly connected** and **strongly connected**



weakly connected

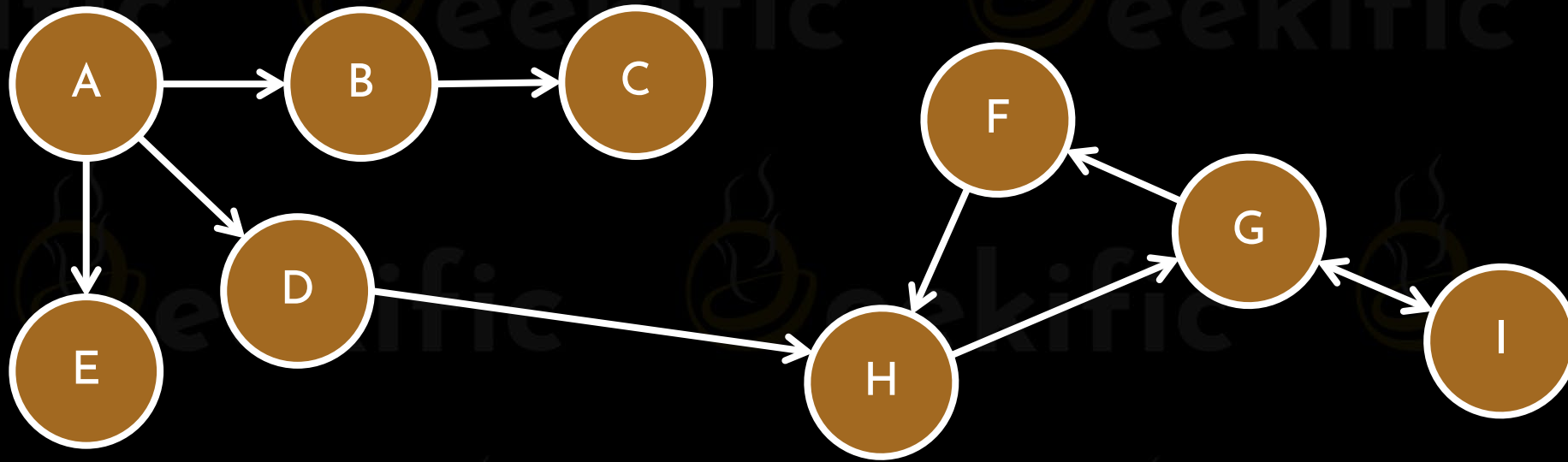
strongly connected



strongly connected



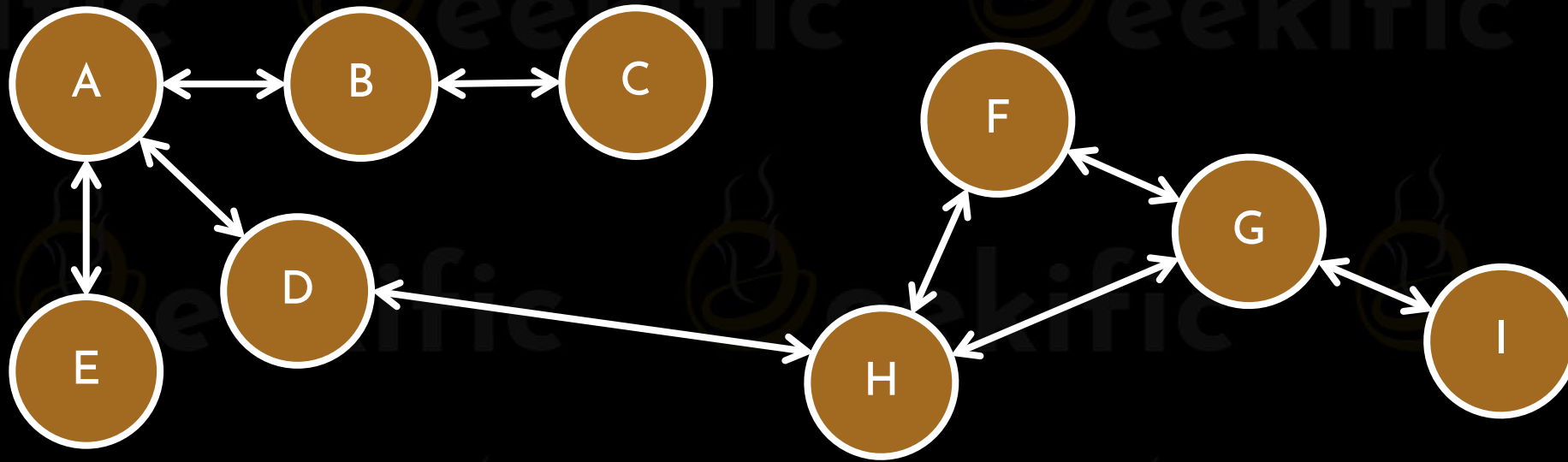
every vertex should be reachable from any other vertex in the graph



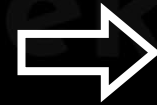
weakly connected



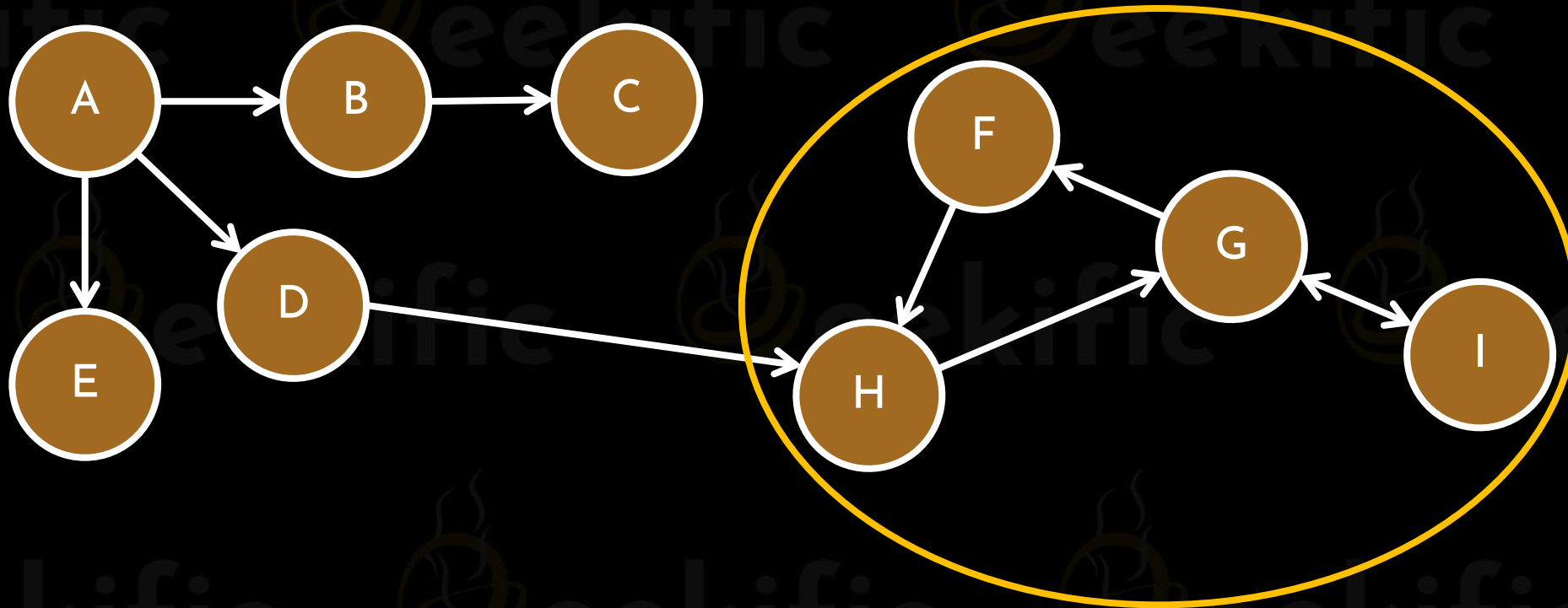
by replacing all directed edges with undirected edges, it becomes a connected undirected graph



**strongly connected  
component**



every vertex is reachable from  
any other vertex from within the  
component



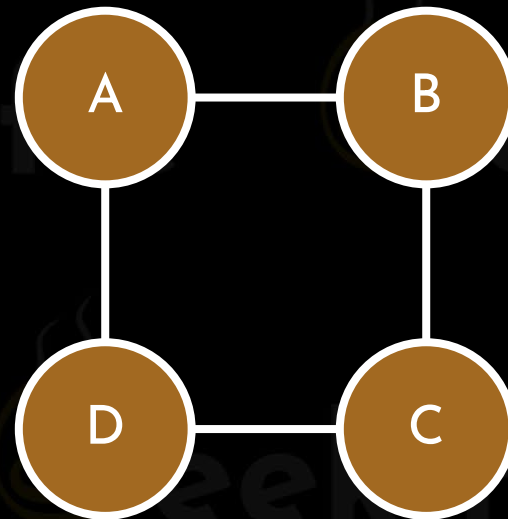
applications: **navigation maps, car routing**, and more



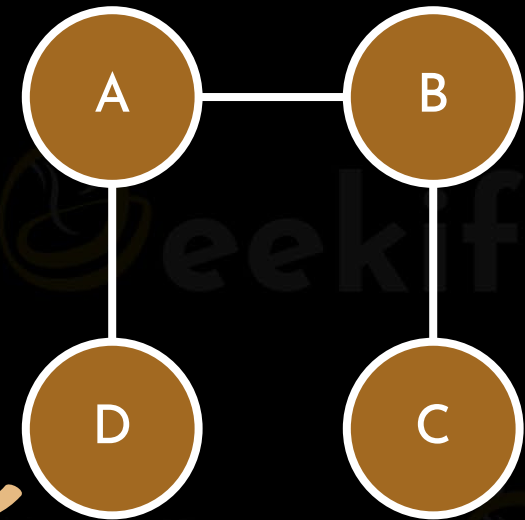
spanning trees

undirected  
acyclic  
connected

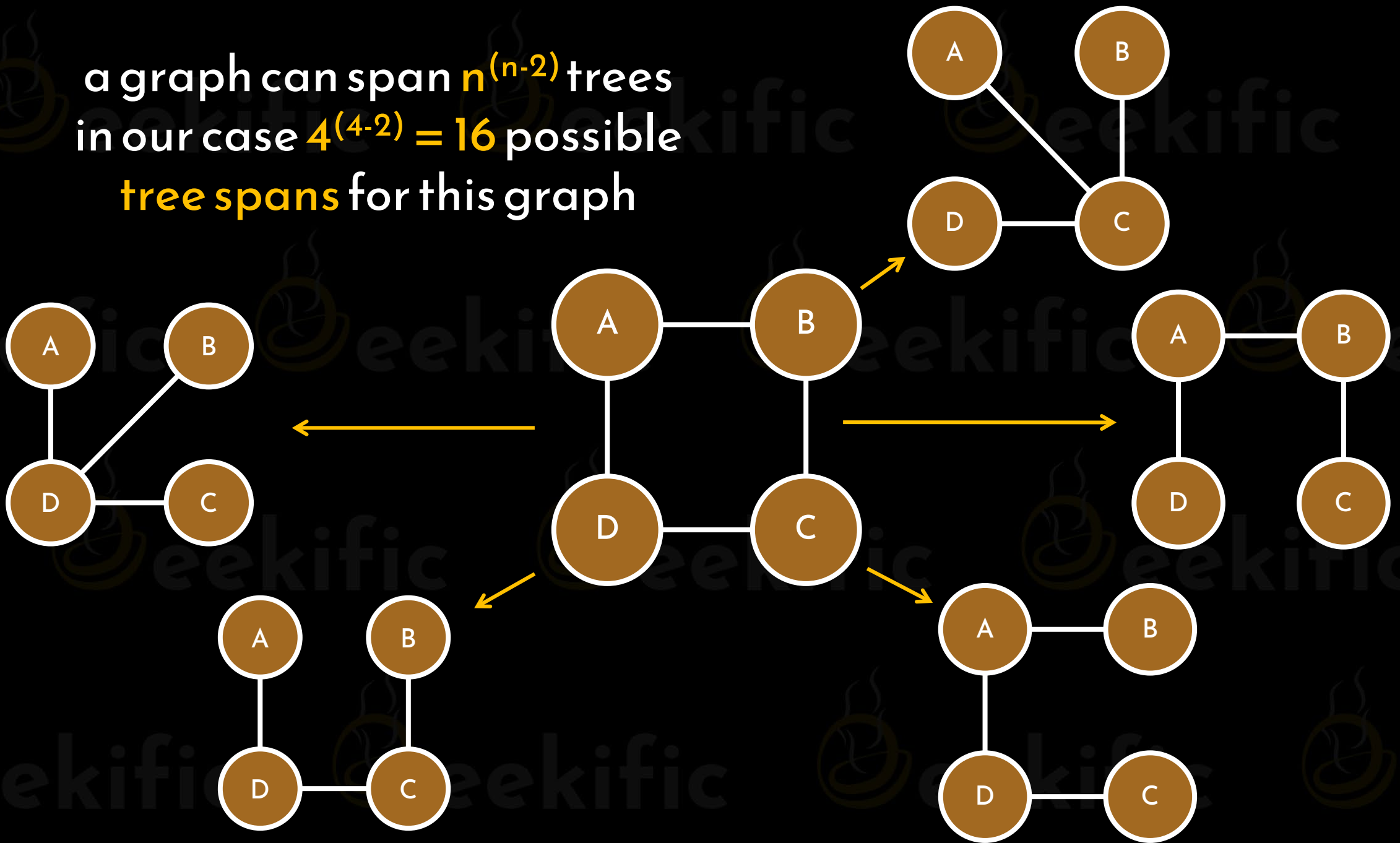
this graph can  
**span** multiple  
trees



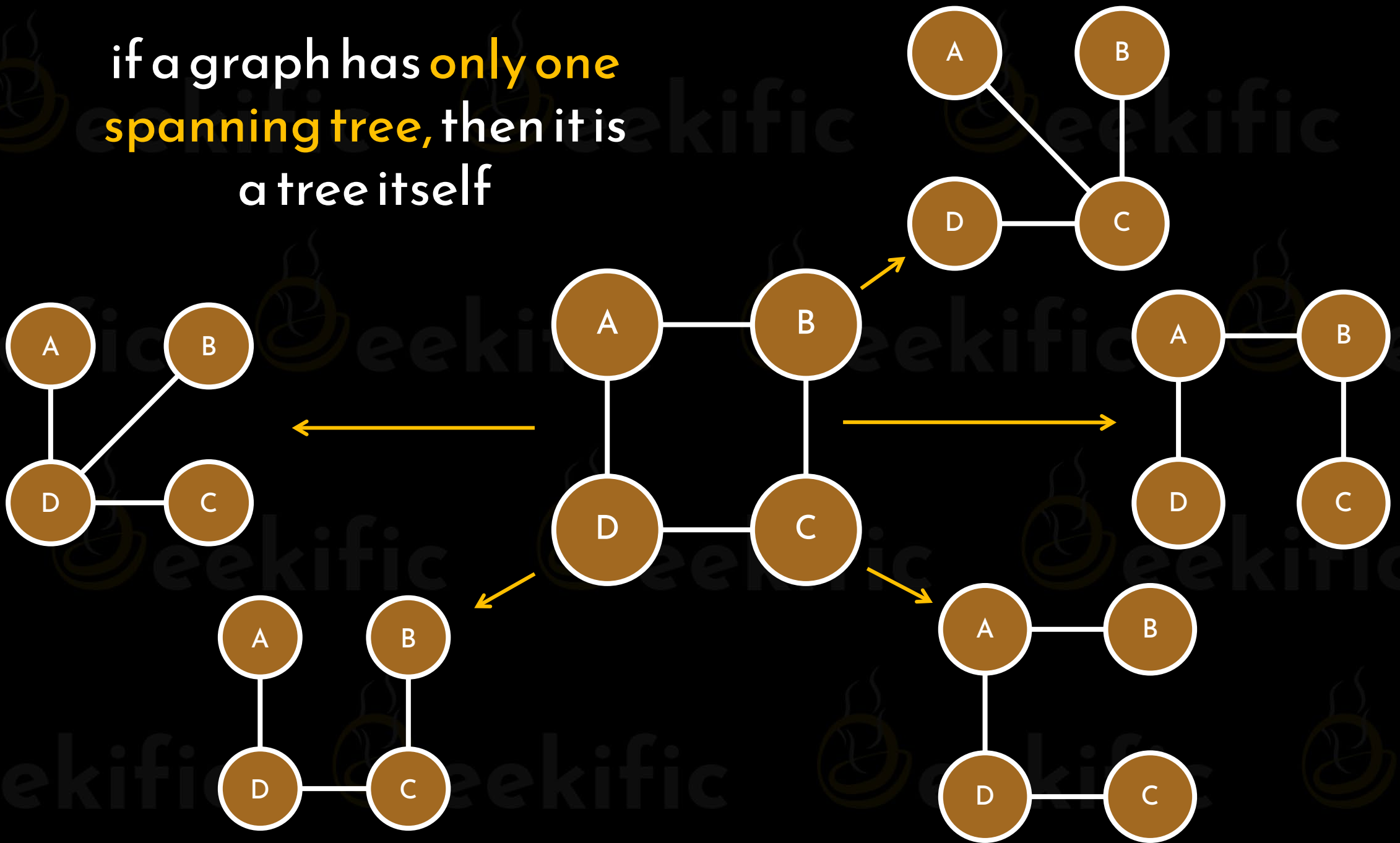
a spanning tree for this graph



a graph can span  $n^{(n-2)}$  trees  
in our case  $4^{(4-2)} = 16$  possible



if a graph has **only one spanning tree**, then it is a tree itself



# Applications

spanning trees have numerous applications:

network routing protocols

cluster analysis

and many more



# Summary

a **graph** is made of **vertices** connected by **edges**  
– not all vertices need to be connected

two types: **undirected** and **directed**, represented  
via **adjacency matrix** or **adjacency list**

**connected** = every vertex reachable from any other  
in directed graphs: **strongly** vs **weakly** connected

**cycle** = the neighbor of my neighbor is my neighbor  
no cycle = **acyclic** – if also undirected and connected, it is a **tree**

a graph can span **spanning trees** – if it spans only one, it is a tree itself

